Image Interpolation Using Neural Networks

Team Member	Registration No.
Suresh Kumar	20233282
Shivam Aryan	20233265
Shreyansh Gangwar	20233270
Santosh Kumar Patra	20234149

Abstract

Classical interpolation cannot restore detail that is lost when an image is downscaled, while neural networks can learn to estimate it. This project compares four classical methods (nearest-neighbour, bilinear, bicubic, Lanczos), a small SRCNN and our own lightweight network, our_nn (about 200 thousand parameters), for ×2 and ×4 super-resolution with and without Gaussian noise. our_nn combines a bicubic skip connection, residual blocks, channel attention and a fixed Sobel and Laplacian edge branch.

All methods are tested on Set5, Set14, BSD100 and Urban100, with PSNR and SSIM computed on the luminance channel. At ×4 on clean images our_nn gains 1.32 dB over bicubic interpolation and 0.93 dB over SRCNN (SSIM 0.764 against 0.714), and at ×2 it gains 2.40 dB over bicubic. With noise ($\sigma = 15$) it is still the best method, 1.11 dB above bicubic.

An ablation study shows that the bicubic skip gives most of the improvement, the edge branch a small gain of about 0.1 dB, and channel attention none. Each configuration was trained once.

1. Introduction

Image interpolation estimates the missing pixels when an image is enlarged, so that a low-resolution (LR) image can be shown at a higher resolution. When the goal is to recover fine detail and not only to resize, the same task is called single-image super-resolution (SR). Classical interpolation (nearest-neighbour, bilinear, bicubic and Lanczos) computes each new pixel as a fixed weighted average of nearby pixels. These methods are fast and need no training, but they cannot restore detail that the LR image has lost, so edges look blurred and, on noisy input, the noise is enlarged together with the picture.

Convolutional neural networks (CNNs) instead learn the mapping from LR to high-resolution (HR) images from examples. SRCNN shows that a three-layer CNN already beats bicubic interpolation. Later work adds residual learning on top of the bicubic image [3], deeper residual blocks without batch normalisation [3], sub-pixel convolution for cheap upscaling and channel attention.

The aim of this project is to measure how much a small network gains over classical interpolation, and to find out which of its ingredients are responsible for the gain. To do this we pursue three objectives:

Objectives.

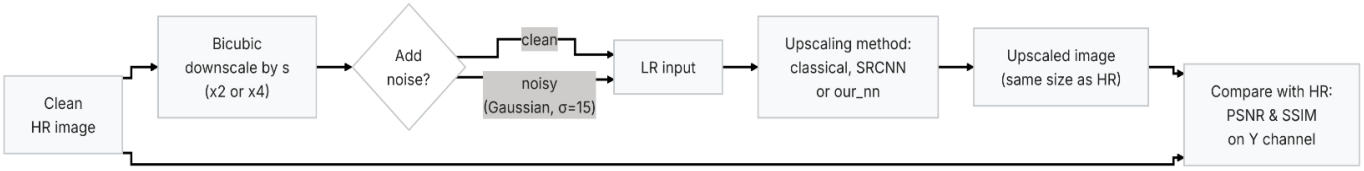
1. Build a reproducible pipeline that degrades clean images (downscaling, optional Gaussian noise), upscales them with different methods and scores the result against the original image.
2. Compare four classical methods, a small SRCNN and our own lightweight network, our_nn, at scale factors ×2 and ×4, with and without noise.
3. Run an ablation study of our_nn to measure what each of its components contributes.

Our Contributions. We design our_nn, a network of about 200 thousand parameters that combines known building blocks: a bicubic skip connection, residual blocks, channel attention and a fixed edge-prior branch built from Sobel and Laplacian filters.

2. Methodology

2.1 Experimental pipeline. Every experiment follows the same steps. A clean HR image is degraded to an LR input: it is downscaled by a factor s (×2 or ×4) with bicubic interpolation and, in the noisy setting, Gaussian noise with standard deviation σ = 15 (on the 0–255 intensity scale) is added after downscaling. A model upscales the LR image back to the original size and the result is compared with the untouched HR image.

The noise of each test image comes from a fixed random seed, so every method receives exactly the same noisy input. Test images are cropped so that their sides are divisible by s. The code is written in PyTorch; models, datasets, degradations and metrics are registered by name, and each experiment is described by one YAML configuration file.



2.2 Data. Training uses random HR patches from DIV2K, four standard benchmarks are used for testing (Table 1).

Table 1. Datasets.

Role	Set	Images	Content
Training	DIV2K train	800	2K-resolution photographs; random patches of [FILL: patch size] × [FILL: patch size] px, random flips and 90° rotations
Validation	Set14	14	used to choose the best checkpoint (it is also a test set, see Section 3.5)
Test	Set5	5	baby, bird, butterfly, head, woman
Test	BSD100	100	natural images: animals, plants, people, food
Test	Urban100	100	buildings and city scenes with many straight edges

2.3 Methods compared. Four classical methods need no training. Nearest-neighbour copies the closest pixel, bilinear averages the 2 × 2 nearest pixels, bicubic fits a cubic polynomial through the 4 × 4 nearest pixels, and Lanczos uses a windowed sinc function over a similar neighbourhood. SRCNN [1] first enlarges the image with bicubic interpolation and then applies three convolution layers (9 × 9 with 64 filters, 1 × 1 with 32 filters, 5 × 5 with 3 filters) that learn to restore detail, it has 20,099 parameters and here works on RGB images.

2.4 The our_nn network. our_nn works at LR resolution and enlarges only at the end, which keeps it cheap (Figure 1). It has five parts, each built from a known technique:

1. **Head and edge branch.** A 3 × 3 convolution turns the 3 colour channels into 32 feature maps. In parallel, a luminance map (0.299 R + 0.587 G + 0.114 B) is filtered with fixed Sobel-x, Sobel-y and Laplacian kernels, which respond to edges. The three edge maps pass through a convolution and are fused with the head features by a 1 × 1 convolution. The three filters are not trained.
2. **Residual blocks.** Six blocks (convolution, ReLU, convolution) each learn a small correction that is added back to the block input (residual learning). There is no batch normalisation, and each correction is multiplied by 0.1 for stable training. A global skip connection adds the head features after the last block.
3. **Channel attention.** Inside each block, a squeeze-and-excitation module averages every feature map to one number, passes the 32 numbers through two small layers and a sigmoid, and rescales the feature maps, so useful channels are emphasised[4].
4. **Sub-pixel upsampler.** A convolution produces s^2 times more channels, and PixelShuffle rearranges them into an image that is s times larger (two ×2 stages for ×4, one stage for ×2).
5. **Bicubic skip.** A final 3 × 3 convolution predicts a residual that is added to the bicubic-upscaled input [3], so the network only has to learn the missing detail. Its weights start at zero, so the untrained network equals bicubic interpolation.

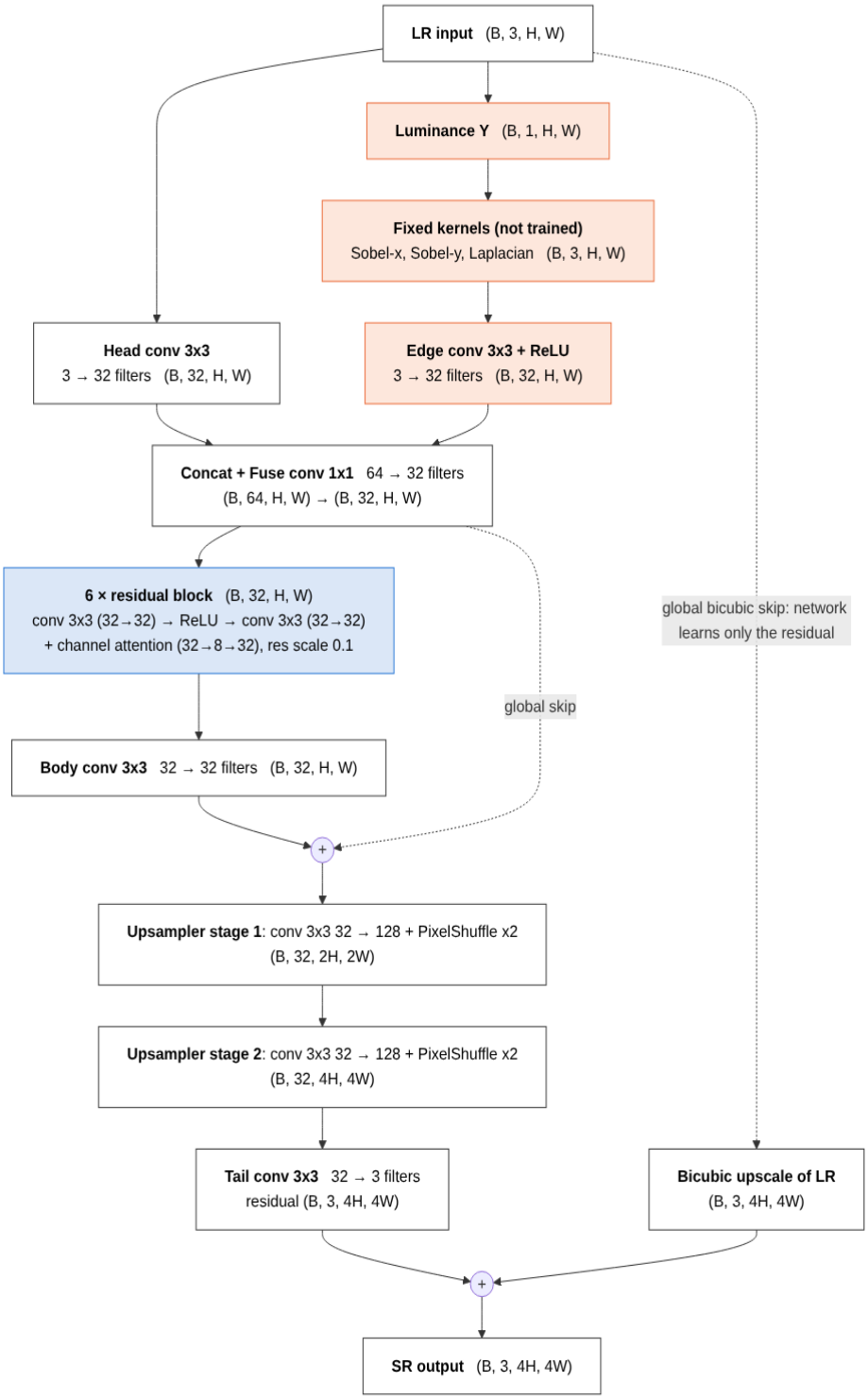


Figure 1 : Block diagram of our_nn (×4). The dashed path is the bicubic skip, the shaded blocks are optional and are switched off in the ablation variants.

The full network has 202,259 parameters at ×4 and 165,267 at ×2. For the ablation study three components are switched off one at a time (Table 4 in Section 3.3).

2.5 Training.

All networks are trained with the L1 loss and the Adam optimiser on random 96 × 96 HR training patches, with random flips and 90° rotations. The learning rate is 2×10^{-4} for our_nn and 1×10^{-4} for SRCNN, with a batch size of 16 for our_nn and 32 for SRCNN. Training runs for 100 epochs, and each epoch passes ten times over the 800 training images, which gives 8,000 patches per epoch. The learning rate is halved every 40 epochs. Every 5 epochs the model is scored on the validation set, and the checkpoint with the highest validation PSNR is kept and used for testing. The noisy models are trained on inputs made by the same degradation as at test time ($\sigma = 15$). All runs use seed 42 and were trained on T4 gpu of Kaggle platform.

2.6 Evaluation. Quality is measured with two standard metrics, computed on the luminance (Y) channel.

$$Y = 16 + 65.481\,R + 128.553\,G + 24.966\,B$$

PSNR (peak signal-to-noise ratio, in dB, higher is better) is based on the mean squared error (MSE) between the result and the HR image:

$$PSNR = 10 \log_{10} (255^2 / MSE)$$

SSIM (structural similarity, from 0 to 1, higher is better) compares local luminance, contrast and structure and agrees better with perceived quality. Scores are averaged over the images of each test set. Where one number per method is shown, it is the mean of the four test-set averages, so each dataset has equal weight.

3. Results and Discussion

our_nn is the best method in every setting we tested: at ×4 on clean images it gains 1.32 dB PSNR over bicubic interpolation and 0.93 dB over our SRCNN, and most of this gain comes from its bicubic skip connection. Unless stated otherwise, numbers are means over the four test sets.

3.1 Comparison at ×4 (clean). Table 2 lists the scores. The ranking is the same on all four test sets: nearest < bilinear < bicubic < Lanczos < SRCNN < our_nn. Lanczos is only 0.11 dB better than bicubic and SRCNN 0.39 dB better, whereas our_nn is 1.32 dB better and raises SSIM from 0.714 to 0.764. The gain over bicubic is smallest on BSD100 (+0.85 dB), whose natural textures are hard to predict, and largest on Set5 (+1.92 dB). The bicubic score on Set5 (28.63 dB) is close to the value usually reported for this protocol, which suggests that the data and metric code are correct. Our SRCNN is weaker than the roughly 30.5 dB on Set5 reported for the original model [1], so "our_nn beats SRCNN" refers to the SRCNN trained in this project.

Table 2. Results at ×4 on clean images (PSNR in dB / mean SSIM in the last column).

Method	Set5	Set14	BSD100	Urban100	Mean PSNR	Mean SSIM
Nearest	26.26	24.74	25.04	22.17	24.55	0.660
Bilinear	27.56	25.52	25.55	22.69	25.33	0.688
Bicubic	28.63	26.22	26.05	23.24	26.04	0.714

Method	Set5	Set14	BSD100	Urban100	Mean PSNR	Mean SSIM
Lanczos	28.81	26.32	26.13	23.33	26.15	0.718
SRCNN	29.19	26.60	26.34	23.58	26.43	0.730
our_nn	**30.56**	**27.56**	**26.90**	**24.41**	**27.36**	**0.764**

3.2 Other scale and noise. Table 3 summarises ×2 and the noisy case. At ×2 our_nn reaches 32.74 dB, 2.40 dB above bicubic, while SRCNN is only 0.15 dB above bicubic and slightly below Lanczos. A plausible reason for the larger network gain at ×2 is that more of the lost detail can still be inferred from the LR image, whereas at ×4 it is largely missing.

Table 3. Mean PSNR (dB) / mean SSIM over the four test sets.

Setting	Nearest	Bilinear	Bicubic	Lanczos	SRCNN	our_nn
×2 clean	28.37 / 0.848	29.04 / 0.843	30.34 / 0.876	30.57 / 0.881	30.49 / 0.877	**32.74 / 0.914**
×4 clean	24.55 / 0.660	25.33 / 0.688	26.04 / 0.714	26.15 / 0.718	26.43 / 0.730	**27.36 / 0.764**
×4 noise σ = 15	23.14 / 0.515	24.46 / 0.624	24.53 / 0.608	24.50 / 0.603	25.26 / 0.672	**25.63 / 0.689**

Noise (σ = 15) lowers every method by 0.9 to 1.7 dB. Among the classical methods, the sharper the kernel the larger the loss (Lanczos −1.65 dB, bilinear −0.87 dB), because sharpening also amplifies noise; bilinear therefore has the best SSIM (0.624 against 0.608 for bicubic) and Lanczos is no longer better than bicubic. The networks trained on noisy data learn to suppress part of the noise: SRCNN rises from 0.39 to 0.73 dB above bicubic, and our_nn is 1.11 dB above bicubic and 0.38 dB above SRCNN, with SSIM 0.689 against 0.608. Its lead over SRCNN is smaller than on clean images (0.93 dB). A possible reason is that the bicubic skip carries the noise to the output, so the network must learn to cancel it, which fits the ablation below: the skip adds only 0.23 dB under noise but 0.62 dB on clean images. We did not test a model trained on clean images and tested on noisy ones.

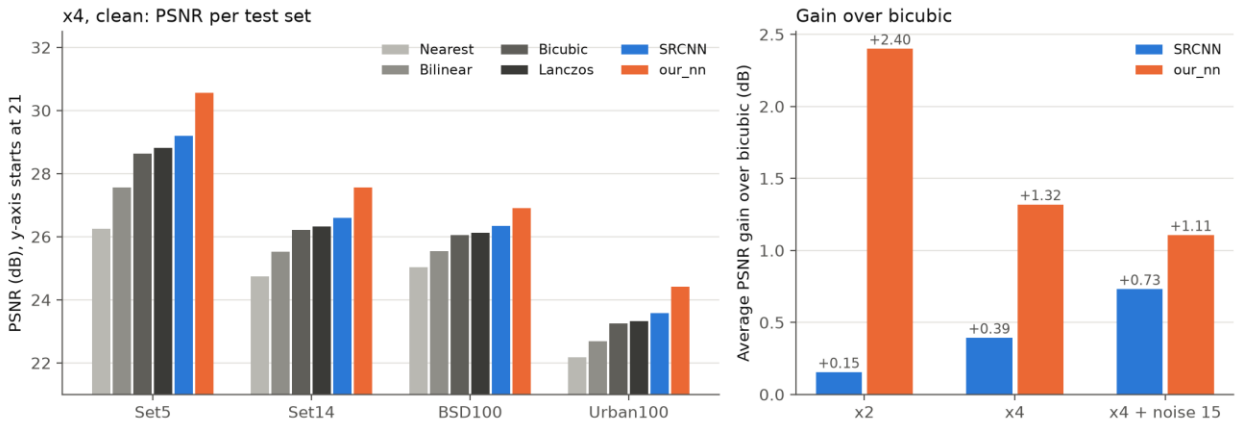


Figure 2. Left: PSNR at ×4 (clean) for each method and test set. Right: PSNR gain over bicubic for SRCNN and our_nn at ×2, ×4 and ×4 with noise.

3.3 Ablation study. Table 4 adds the components of our_nn one at a time (×4, mean over the four test sets).

Table 4. Ablation of our_nn (PSNR in dB; change relative to the previous row).

Variant	Bicubic skip	Attention	Edge branch	Clean PSNR	Change	Noisy PSNR	Change	Parameters
A0 (plain)	no	no	no	26.63		25.36		195,971
A1 (+skip)	yes	no	no	27.26	**+0.62**	25.59	**+0.23**	195,971
A2 (+attention)	yes	yes	no	27.25	−0.01	25.55	−0.04	199,283
FULL our_nn (+edge branch)	yes	yes	yes	**27.36**	+0.11	25.63	+0.09	202,259

- **The bicubic skip is the main contributor** (+0.62 dB clean, +0.23 dB noisy). Even without it, the plain network is 0.60 dB above bicubic and 0.20 dB above SRCNN, so the deep residual body is useful by itself.
- **Channel attention did not help** (−0.01 and −0.04 dB for 3,312 extra parameters). With a model this small and short training it can be left out.
- **The edge branch adds a little**: +0.11 dB clean and +0.09 dB noisy for 2,976 parameters, positive on all four test sets in both settings. The effect is small and comes from one run per configuration. Its convolution layers are trainable, so we cannot tell whether the Sobel and Laplacian prior or the extra capacity is responsible

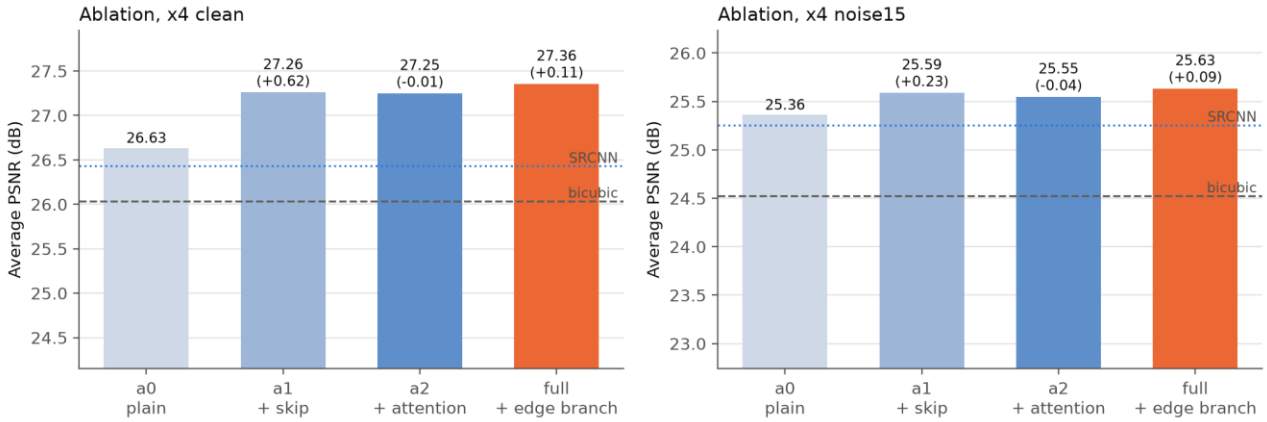


Figure 3. Contribution of each component to the PSNR, clean (left) and noisy (right) input.

3.4 Cost. SRCNN has 20,099 parameters and our_nn 202,259, yet on the large Urban100 images (×4) SRCNN needs 31.8 ms per image and our_nn 24.9 ms, whereas on the small Set5 images SRCNN is faster (7.3 ms against 11.0 ms) .

Speed depends on the amount of computation, not only on the number of parameters: SRCNN works at HR resolution, so its cost grows quickly with image size, whereas our_nn works at LR resolution and

enlarges only at the end. Single timings on a shared GPU are noisy, so these are rough figures. The optional components raise the time from 21.0 ms (plain) to 24.9 ms (full).

3.5 Qualitative results. Figure 4 compares enlarged crops of one test image.



Figure 4. Crops of the HR image, the LR input and the results of bicubic, SRCNN and our_nn.

4. Conclusion

We built a reproducible benchmark and showed that small neural networks beat classical interpolation under the same test protocol. Our network our_nn, with about 200 thousand parameters, is 1.32 dB better than bicubic interpolation at ×4, 2.40 dB better at ×2 and 1.11 dB better when noise ($\sigma = 15$) is added; it is also better than our SRCNN in every setting. The ablation study shows which design choices matter: residual learning on top of the bicubic image gives most of the gain (+0.62 dB), channel attention gives none at this model size, and the fixed edge branch gives a small gain of about 0.1 dB that needs further testing. Classical methods lose quality under noise because sharper kernels amplify it, while networks trained on noisy data partly remove it.

Future work can expand to repeating each run with several seeds and reporting the spread, a control experiment for the edge branch (a trainable filter bank of the same size), testing models trained on clean images with noisy input and with real degradations, longer training of larger models, and adding a perceptual metric such as LPIPS.

References

1. C. Dong, C. C. Loy, K. He, and X. Tang, "Image super-resolution using deep convolutional networks," IEEE Trans. Pattern Anal. Mach. Intell., vol. 38, no. 2, pp. 295–307, 2016.
2. J. Kim, J. K. Lee, and K. M. Lee, "Accurate image super-resolution using very deep convolutional networks," in Proc. IEEE CVPR, 2016, pp. 1646–1654.
3. B. Lim, S. Son, H. Kim, S. Nah, and K. M. Lee, "Enhanced deep residual networks for single image super-resolution," in Proc. IEEE CVPR Workshops, 2017, pp. 136–144.
4. J. Hu, L. Shen, and G. Sun, "Squeeze-and-excitation networks," in Proc. IEEE CVPR, 2018, pp. 7132–7141.